

```

Stack:-
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;
void push()
{
    int x;
    if(top == MAX-1)
        printf("Stack Overflow\n");
    else
    {
        printf("Enter element: ");
        scanf("%d",&x);
        stack[++top] = x;
    }
}

void pop()
{
    if(top == -1)
        printf("Stack Underflow\n");
    else
        printf("Deleted element = %d\n", stack[top--]);
}

void display()
{
    int i;
    if(top == -1)
        printf("Stack Empty\n");
    else
    {
        for(i=top;i>=0;i--)
            printf("%d\n", stack[i]);
    }
}

int main()
{
    int ch;
    while(1)
    {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d",&ch);

        switch(ch)
        {

```

```

        case 1: push(); break;
        case 2: pop(); break;
        case 3: display(); break;
        case 4: return 0;
    }
}

Queue:-
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;

void push()
{
    int x;
    if(top == MAX-1)
        printf("Stack Overflow\n");
    else
    {
        printf("Enter element: ");
        scanf("%d",&x);
        stack[++top] = x;
    }
}

void pop()
{
    if(top == -1)
        printf("Stack Underflow\n");
    else
        printf("Deleted element = %d\n", stack[top--]);
}

void display()
{
    int i;
    if(top == -1)
        printf("Stack Empty\n");
    else
    {
        for(i=top;i>=0;i--)
            printf("%d\n", stack[i]);
    }
}

int main()
{
    int ch;
    while(1)
    {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d",&ch);

        switch(ch)
        {

```

```

        case 1: insert(); break;
        case 2: delete(); break;
        case 3: display(); break;
        case 4: return 0;
    }
}

4. DFS
#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v)
{
    int i;

    visited[v]=1;
    printf("%d ",v);

    for(i=1;i<=n;i++)
    {
        if(a[v][i] && !visited[i])
            dfs(i);
    }
}

int main()
{
    int i,j,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d",&start);
    dfs(start);
    return 0;
}

5. BFS
#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v)
{
    int i;

    visited[v]=1;
    printf("%d ",v);

    for(i=1;i<=n;i++)
    {
        if(a[v][i] && !visited[i])
            dfs(i);
    }
}

int main()
{
    int i,j,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d",&start);
    dfs(start);

    return 0;
}

6. Topological Sorting
#include <stdio.h>

int a[10][10], indegree[10], n;


```

```

int a[100],n,i;

printf("Enter n: ");
scanf("%d",&n);

for(i=0;i<n;i++)
    scanf("%d",&a[i]);

mergesort(a,0,n-1);

printf("Sorted elements:\n");

for(i=0;i<n;i++)
    printf("%d ",a[i]);

return 0;
}

8. Quick Sort
#include <stdio.h>
void quicksort(int a[], int low, int high)
{
    int i,j,pivot,temp;

    if(low<high)
    {
        pivot=a[low];
        i=low;
        j=high;

        while(i<j)
        {
            while(a[j]<=pivot && i<high)
                i++;

            while(a[i]>pivot)
                j--;

            if(i<j)
            {
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
        }

        a[low]=a[i];

```

```

        case 1: push(); break;
        case 2: pop(); break;
        case 3: display(); break;
        case 4: return 0;
    }
}

Queue:-
#include <stdio.h>
#define MAX 5
int stack[MAX], top = -1;

void push()
{
    int x;
    if(top == MAX-1)
        printf("Stack Overflow\n");
    else
    {
        printf("Enter element: ");
        scanf("%d",&x);
        stack[++top] = x;
    }
}

void pop()
{
    if(top == -1)
        printf("Stack Underflow\n");
    else
        printf("Deleted element = %d\n", stack[top--]);
}

void display()
{
    int i;
    if(top == -1)
        printf("Stack Empty\n");
    else
    {
        for(i=top;i>=0;i--)
            printf("%d\n", stack[i]);
    }
}


```

```

        case 1: insert(); break;
        case 2: delete(); break;
        case 3: display(); break;
        case 4: return 0;
    }
}

4. DFS
#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v)
{
    int i;

    visited[v]=1;
    printf("%d ",v);

    for(i=1;i<=n;i++)
    {
        if(a[v][i] && !visited[i])
            dfs(i);
    }
}

int main()
{
    int i,j,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d",&start);
    dfs(start);
    return 0;
}

5. BFS
#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v)
{
    int i;

    visited[v]=1;
    printf("%d ",v);

    for(i=1;i<=n;i++)
    {
        if(a[v][i] && !visited[i])
            dfs(i);
    }
}

int main()
{
    int i,j,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d",&start);
    dfs(start);

    return 0;
}

6. Topological Sorting
#include <stdio.h>

int a[10][10], indegree[10], n;


```

7. Merge Sort

```

#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
    int i,j,k,h[100];

    i=low;
    j=mid+1;
    k=low;

    while(i<=mid && j<=high)
    {
        if(a[i]<a[j])
            b[k++]=a[i++];
        else
            b[k++]=a[j++];
    }

    while(i<=mid)
        b[k++]=a[i++];

    while(j<=high)
        b[k++]=a[j++];

    for(i=low;j<=high;j++)
        a[i]=b[i];
}

void mergesort(int a[], int low, int high)
{
    int mid;

    if(low<high)
    {
        mid=(low+high)/2;

        mergesort(a,low,mid);
        mergesort(a,mid+1,high);

        merge(a,low,mid,high);
    }
}

int main()
{

```

```

int main()
{
    int ch;
    while(1)
    {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d",&ch);

        switch(ch)
        {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: return 0;
        }
    }
}

Circular Queue:-
#include <stdio.h>
#define MAX 5

int cq[MAX];
int front=-1, rear=-1;

void insert()
{
    int x;

    if((rear+1)%MAX == front)
    {
        printf("Queue Full\n");
        return;
    }

    printf("Enter element: ");
    scanf("%d",&x);

    if(front==-1)
        front=0;

    rear=(rear+1)%MAX;
    cq[rear]=x;
}


```

5. BFS

```

#include <stdio.h>

int a[10][10], visited[10], n;

void dfs(int v)
{
    int i;

    visited[v]=1;
    printf("%d ",v);

    for(i=1;i<=n;i++)
    {
        if(a[v][i] && !visited[i])
            dfs(i);
    }
}

int main()
{
    int i,j,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter adjacency matrix\n");

    for(i=1;i<=n;i++)
        for(j=1;j<=n;j++)
            scanf("%d",&a[i][j]);

    printf("Enter starting vertex: ");
    scanf("%d",&start);
    dfs(start);

    return 0;
}


```

6. Topological Sorting

```

#include <stdio.h>

int a[10][10], indegree[10], n;


```

```

int a[100],n,i;

printf("Enter n: ");
scanf("%d",&n);

for(i=0;i<n;i++)
    scanf("%d",&a[i]);

mergesort(a,0,n-1);

printf("Sorted elements:\n");

for(i=0;i<n;i++)
    printf("%d ",a[i]);

return 0;
}

8. Quick Sort
#include <stdio.h>
void quicksort(int a[], int low, int high)
{
    int i,j,pivot,temp;

    if(low<high)
    {
        pivot=a[low];
        i=low;
        j=high;

        while(i<j)
        {
            while(a[j]<=pivot && i<high)
                i++;

            while(a[i]>pivot)
                j--;

            if(i<j)
            {
                temp=a[i];
                a[i]=a[j];
                a[j]=temp;
            }
        }

        a[low]=a[i];

```

```

        a[j]=pivot;
        quicksort(a,low,j-1);
        quicksort(a,j+1,high);
    }

int main()
{
    int a[100],n,i;

    printf("Enter n: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    quicksort(a,0,n-1);

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}

```

9. Heap Sort

```

#include <stdio.h>

void quicksort(int a[], int low, int high)
{
    int i,j,pivot,temp;

    if(low<high)
    {
        pivot=a[low];
        i=low;
        j=high;

        while(i<j)
        {
            while(a[i]<=pivot && i<high)
                i++;

            while(a[j]>pivot)
                j--;

            swap(a[i],a[j]);
        }

        swap(a[i],a[j]);
    }

    quicksort(a,low,i-1);
    quicksort(a,i+1,high);
}

int main()
{
    int a[100],n,i;

    printf("Enter n: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    quicksort(a,0,n-1);

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}

```

```

scanf("%d",&n);

for(i=0;i<n;i++)
    scanf("%d",&a[i]);

heapsort(a,n);

for(i=0;i<n;i++)
    printf("%d ",a[i]);

return 0;
}

```

```

if(!visited[v]==0)
{
    printf("%d -> %d = %d\n",u,v,min);
    total+=min;
    visited[v]=1;
    ne++;
}

a[u][v]=a[v][u]=999;

printf("Minimum cost = %d",total);

return 0;
}

```

13. Kruskal's Algorithm

```

#include <stdio.h>

int parent[10];

int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}

int uni(int i,int j)
{
    if(i==j)
    {
        parent[i]=i;
        return 1;
    }
    return 0;
}

int main()
{
    int a[10][10], n, i, j;
    int min, mincost=0, ne=1, u, v;

    printf("Enter vertices: ");
    scanf("%d",&n);
    printf("Enter cost matrix:\n");
    for(i=1;i<n;i++)
    {

```

```

        if(i<j)
        {
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }

        a[low]=a[i];
        a[j]=pivot;

        quicksort(a,low,j-1);
        quicksort(a,j+1,high);
    }
}

int main()
{
    int a[100],n,i;

    printf("Enter n: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    quicksort(a,0,n-1);

    for(i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
}

```

11. Floyd Warshall Algorithm

```

#include <stdio.h>

int main()
{
    int a[10][10], n, i, j, k;

    printf("Enter vertices: ");
    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<n;i++)
        for(j=1;j<n;j++)
            scanf("%d",&a[i][j]);

    for(k=1;k<n;k++)
    {
        for(i=1;i<n;i++)
        {
            for(j=1;j<n;j++)
            {
                if(a[i][j] > a[i][k] + a[k][j])
                    a[i][j] = a[i][k] + a[k][j];
            }
        }
    }

    printf("Shortest path matrix:\n");

    for(i=1;i<n;i++)
    {
        for(j=1;j<n;j++)
            printf("%d ",a[i][j]);

        printf("\n");
    }

    return 0;
}

```

```

for(j=1;j<n;j++)
{
    scanf("%d",&a[i][j]);
    if(a[i][j]==0)
        a[i][j]=999;
}

while(ne<n)
{
    min=999;
    for(i=1;i<n;i++)
    {
        for(j=1;j<n;j++)
        {
            if(a[i][j]<min)
            {
                min=a[i][j];
                u=i;
                v=j;
            }
        }
    }
    j=find(u);
    i=find(v);
    if(uni(i,j))
    {
        printf("%d edge (%d,%d) = %d\n",ne++,u,v,min);
        mincost+=min;
    }
    a[u][v]=a[v][u]=999;
    printf("Minimum cost = %d",mincost);
    return 0;
}

```

10. 0/1 Knapsack

```

#include <stdio.h>

void heapify(int a[], int n, int i)
{
    int largest=i, left=2*i+1, right=2*i+2, temp;

    if(left<n && a[left]>a[largest])
        largest=left;

    if(right<n && a[right]>a[largest])
        largest=right;

    if(largest!=i)
    {
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;

        heapify(a,n,largest);
    }
}

void heapsort(int a[], int n)
{
    int i,temp;

    for(i=n/2-1;i>=0;i--)
        heapify(a,n,i);

    for(i=n-1;i>0;i--)
    {
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;

        heapify(a,0,i);
    }
}

int main()
{
    int a[100],n,i;

```

12. Prim's Algorithm

```

#include <stdio.h>

int main()
{
    int a[10][10], visited[10]={0};
    int n, i, j, ne=1, min, u, v, total=0;

    printf("Enter vertices: ");
    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<n;i++)
        for(j=1;j<n;j++)
        {
            scanf("%d",&a[i][j]);

            if(a[i][j]==0)
                a[i][j]=999;
        }

    visited[1]=1;

    while(ne<n)
    {
        min=999;

        for(i=1;i<n;i++)
        {
            for(j=1;j<n;j++)
            {
                if(a[i][j]<min && visited[j]==0)
                {
                    min=a[i][j];
                    u=i;
                    v=j;
                }
            }
        }

        visited[v]=1;
        ne++;
    }

    printf("Shortest path matrix:\n");

    for(i=1;i<n;i++)
    {
        for(j=1;j<n;j++)
            printf("%d ",a[i][j]);

        printf("\n");
    }

    return 0;
}

```

14. Fractional Knapsack

```

#include <stdio.h>
int parent[10];
int find(int i)
{
    while(parent[i])
        i=parent[i];
    return i;
}

int uni(int i,int j)
{
    if(i==j)
    {
        parent[i]=i;
        return 1;
    }
    return 0;
}

int main()
{
    int a[10][10], n, i, j;
    int min, mincost=0, ne=1, u, v;
    printf("Enter vertices: ");
    scanf("%d",&n);
    printf("Enter cost matrix:\n");
    for(i=1;i<n;i++)
    {
        for(j=1;j<n;j++)
        {
            scanf("%d",&a[i][j]);

            if(a[i][j]==0)
                a[i][j]=999;
        }
    }

    while(ne<n)
    {
        min=999;
        for(i=1;i<n;i++)
        {
            for(j=1;j<n;j++)
            {
                if(a[i][j]<min)
                {
                    min=a[i][j];
                    u=i;
                    v=j;
                }
            }
        }

        j=find(u);
        i=find(v);
        if(uni(i,j))
        {
            printf("%d edge (%d,%d) = %d\n",ne++,u,v,min);
            mincost+=min;
        }
        a[u][v]=a[v][u]=999;
        printf("Minimum cost = %d",mincost);
        return 0;
    }
}

```

```

        vj;
    }
}

i=find(u);
j=find(v);

if(unl(i,j))
{
    printf("%d edge (%d,%d) = %d\n",ne++,u,v,min);
    mincost+=min;
}

a[u][v]=a[v][u]=999;
}

printf("Minimum cost = %d",mincost);

return 0;
}

15. Dijkstra Algorithm
#include <stdio.h>

int main()
{
    int cost[10][10], dist[10], visited[10];
    int n,i,j,count,min,next,start;

    printf("Enter number of vertices: ");
    scanf("%d",&n);

    printf("Enter cost matrix:\n");

    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            scanf("%d",&cost[i][j]);

            if(cost[i][j]==0)
                cost[i][j]=999;
        }
    }

    printf("Enter source vertex: ");

```

```

    scanf("%d",&start);

    for(i=1;i<=n;i++)
    {
        dist[i]=cost[start][i];
        visited[i]=0;
    }

    dist[start]=0;
    visited[start]=1;
    count=1;

    while(count<n)
    {
        min=999;

        for(i=1;i<=n;i++)
        {
            if(dist[i]<min && !visited[i])
            {
                min=dist[i];
                next=i;
            }
        }

        visited[next]=1;

        for(i=1;i<=n;i++)
        {
            if(!visited[i] && min+cost[next][i]<dist[i])
                dist[i]=min+cost[next][i];
        }

        count++;
    }

    printf("Shortest distances:\n");

    for(i=1;i<=n;i++)
        printf("%d -> %d = %d\n",start,i,dist[i]);

    return 0;
}

```

16. N Queen Problem

```

#include <stdio.h>
#include <stdlib.h>
int x[10], count=0;
int place(int k,int i)
{
    int j;
    for(j=1;j<=k;j++)
    {
        if(x[j]==i || abs(x[j]-i)==abs(j-k))
            return 0;
    }
    return 1;
}

void nqueen(int k,int n)
{
    int i,j;

    for(i=1;i<=n;i++)
    {
        if(place(k,i))
        {
            x[k]=i;
            if(k==n)
            {
                count++;
                printf("\nSolution %d:\n",count);
                for(j=1;j<=n;j++)
                    printf("%d ",x[j]);
            }
            else
                nqueen(k+1,n);
        }
    }
}

int main()
{
    int n;
    printf("Enter n: ");
    scanf("%d",&n);
    nqueen(1,n);
    return 0;
}

```